



MATURI VENKATA SUBBA RAO (MVSr) ENGINEERING COLLEGE
NADERGUL, HYDERABAD-501510

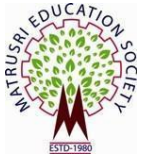
(Sponsored by Matrusri Education Society, Estd.1980)

An Autonomous Institution

Approved by AICTE & Affiliated to Osmania University, Estd.1981

ISO 9001:2015 Certified Institution, Accredited by NAAC

website: www.mvsrec.edu.in



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DATABASE MANAGEMENT SYSTEMS LAB (U21PC381CS)

B.E- III SEM

Acad. Yr: 2024-25

Common to:

CSE (A,B,C,D), Allied Branches (AIML,DS,IOT,CSIT)

CONTENTS

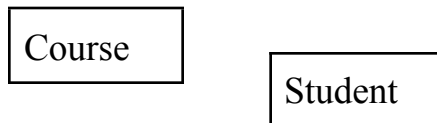
Sl.no	Name of the Topic	Date of Conduct
1	Introduction to E-R Model	
2	Working with XAMPP tool	
3	Introduction to SQL Working with DDL Commands	
4	Working with DML Commands, WHERE clause	
5	Working with Integrity Constraints	
6	Built-in Functions: math, date functions	
7	Usage of Operators: BETWEEN, LIKE, IS NULL, LIMIT, DISTINCT, IN, NOT IN, CASE	
8	Working with Set Operators and JOINS	
9	Working with Views	
10	Usage of ORDER BY, GROUP BY, HAVING and Aggregate functions	
11	Working with Subqueries	
12	PL/SQL : Procedures	
13	PL/SQL : Functions	
14	PL/SQL : Triggers	
15	PL/SQL : Cursors	
16	NoSQL:Basic CRUD operations using MongoDB	

1. INTRODUCTION TO E-R MODEL

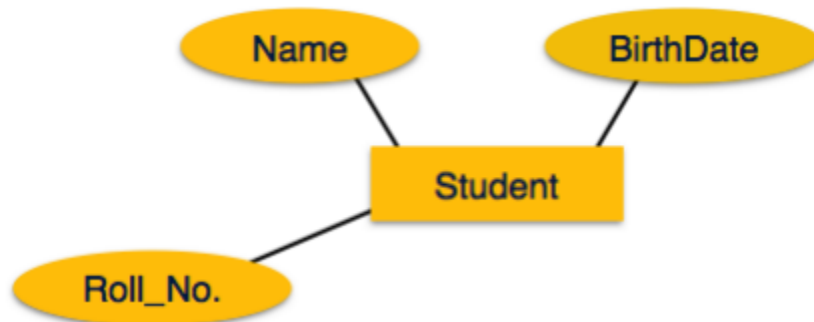
The ER model defines the conceptual view of a database. It works around real-world entities and the associations among them. At view level, the ER model is considered a good option for designing databases.

Components of ER-DIAGRAM:

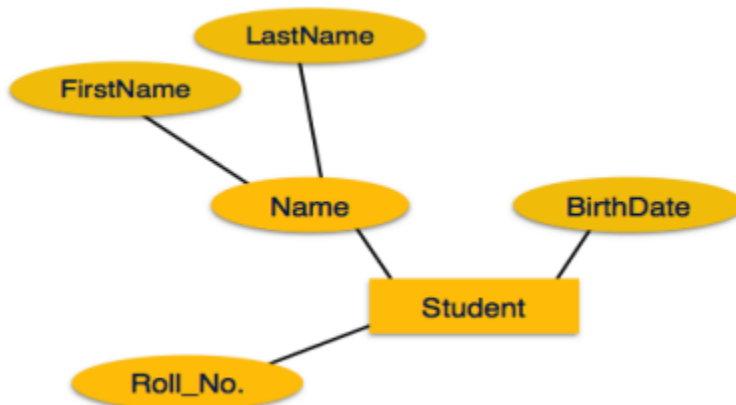
Entity: Rectangles are used to represent entities. The rectangles are provided with names that correspond to the entity set they represent.



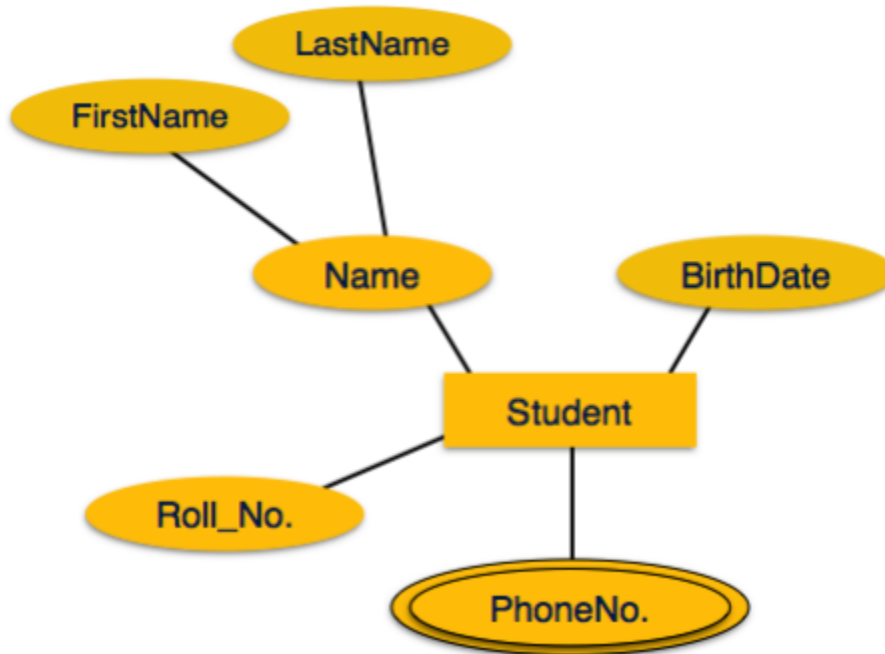
Attributes: Attributes are the properties of entities. Attributes are represented by means of ellipses.



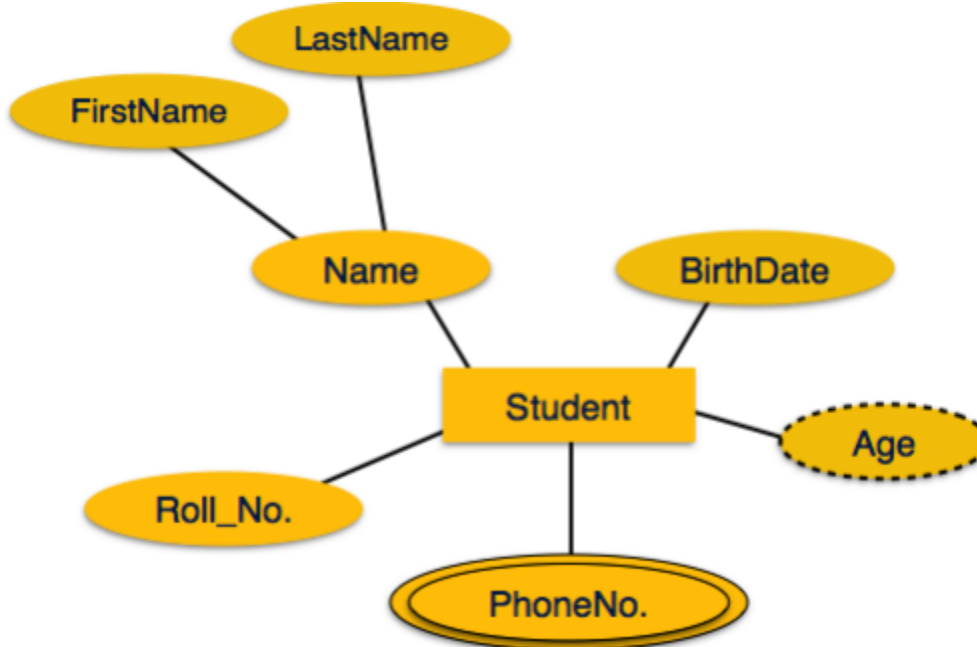
If the attributes are composite, they are further divided in a tree-like structure. Every node is then connected to its attribute. That is, composite attributes are represented by ellipses that are connected with an Ellipse.



→ **Multivalued attributes** are depicted by double ellipses.



→ **Derived attributes** are depicted by dashed ellipse



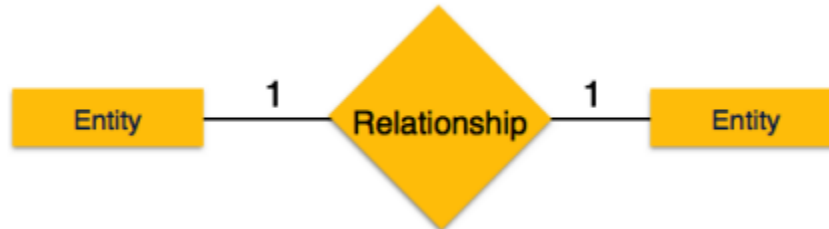
Relationship

Relationships are represented by a diamond-shaped box. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.

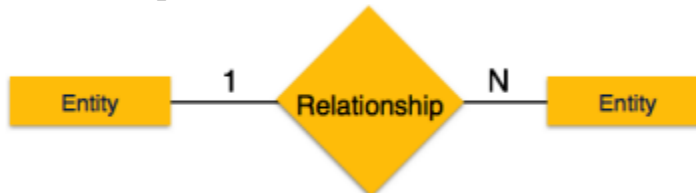
Binary Relationship and Cardinality

A relationship where two entities are participating is called a binary relationship. Cardinality is the number of instances of an entity from a relation that can be associated with the relation.

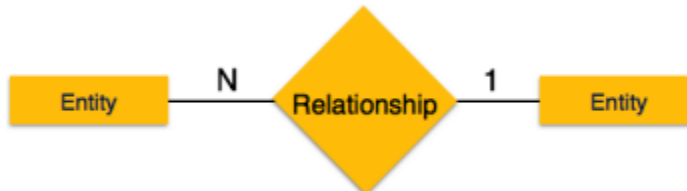
One-to-one – when only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one instance of each entity should be associated with the relationship. It depicts a one-to-one relationship.



One-to-many – when more than one instance of an entity is associated with a relationship, it is marked as '1: N'. The following image reflects that only one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts a one-to-many relationship.



Many-to-one – when more than one instance of an entity is associated with the relationship, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts a many-to-one relationship.



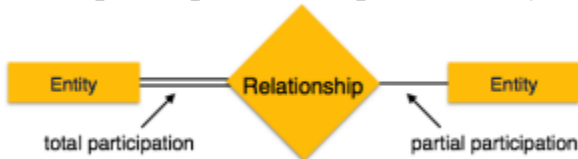
Many-to-many – The following image reflects that more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship. It depicts many-to-many relationships.



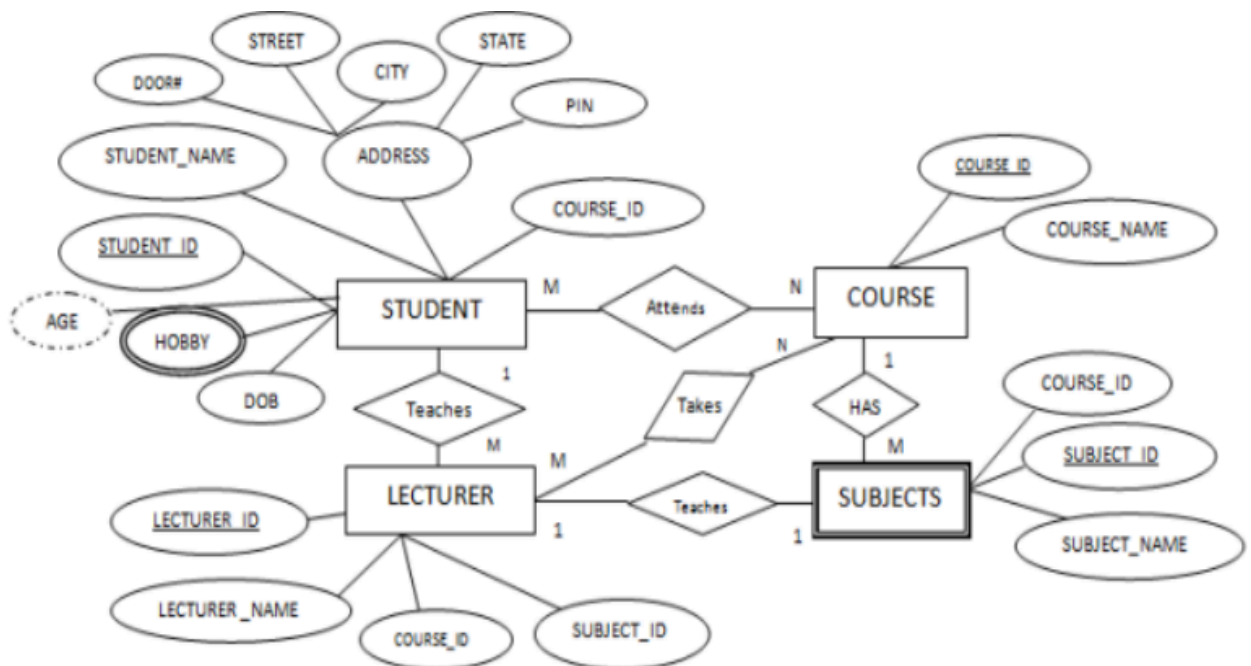
Participation Constraints

Total Participation – each entity is involved in the relationship. Total participation is represented by double lines.

Partial participation – Not all entities are involved in the relationship. Partial participation is represented by single lines.



E-R Diagram for STUDENT INFORMATION SYSTEM



2. WORKING WITH XAMPP TOOL

XAMPP:

It is a cross platform, free and open source software that offers Four Services to the users.

- a) MySQL database
- b) PHP programming
- c) Perl Programming
- d) Apache web server

Among the four, we use MySQL for running SQL and PL/SQL commands. Below are the steps for launching XAMPP and running MySQL in Ubuntu OS.

Step-1: Open Terminal

Step-2: Start the XAMPP Server using below command

sudo /opt/lampp/lampp start

Step-3: Enter into MySQL Command mode using below command

mysql --host=127.0.0.1 --user=root

```
mvsr@u22-1:~$ sudo /opt/lampp/lampp start
[sudo] password for mvsr:
Starting XAMPP for Linux 8.2.12-0...
XAMPP: Starting Apache.../opt/lampp/share/xampp/xampplib: line 22: netstat: command not found
/opt/lampp/share/xampp/xampplib: line 22: netstat: command not found
ok.
XAMPP: Starting MySQL.../opt/lampp/share/xampp/xampplib: line 22: netstat: command not found
ok.
XAMPP: Starting ProFTPD.../opt/lampp/share/xampp/xampplib: line 22: netstat: command not found
ok.
mvsr@u22-1:~$ mysql --host=127.0.0.1 --user=root
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 5.5.5-10.4.32-MariaDB Source distribution

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

After entering into mysql command mode, type the below commands.

i) Display the existing databases;

show databases;

ii) Create a database with the last three digits of the student roll number as shown below. (Replace xxx with last three digits of roll number)

create database csexxx;

iii) Enter into the database using the below command.

use csexxx;

iv) Display the existing tables

show tables;

3. INTRODUCTION TO SQL

What is SQL?

Structured Query Language is a standard query language for creating, storing, accessing and manipulating data in the databases;

It has various types of commands that allows users to access and manipulate databases and data.

Below are the important SQL commands.

- a) **DDL commands** – Used to create/modify the structure of a table (schema of table)

CREATE --- to create a table/database/index

ALTER ---- to modify the table (add/remove/rename/modify columns in table)

RENAME ---- to change the name of an existing table

TRUNCATE ---- to clear all the data inside a table and make the table empty.

DROP ---- To delete the existing table and all its data from the database.

- b) **DML (Data Manipulation Language) commands**: Used to insert/modify/remove/update/fetch the data inside a table

INSERT INTO ---- To insert a row(s) of data in a table

UPDATE ---- To modify the data of row(s) in a table

DELETE ---- To remove the data of row(s) in a table

SELECT ---- To fetch and display the data of row(s) in a table

- c) **DCL (Data Control Language) commands** – Used to create new users and give or remove them specific privileges(roles)

GRANT – To give user privileges to specific users

REVOKE – To remove the assigned privileges from specific users.

- d) **TCL (Transaction Control Language) commands** – used for managing and controlling the transactions in a database to maintain consistency. And it also helps a user manage all the changes made by the DML commands for maintaining its transactions.

COMMIT – To apply changes permanently in the database.

ROLLBACK – To revoke or undo the changes done in the database

SAVEPOINT – To apply changes temporarily with a name and allow users to jump to that change when required.

DDL Commands:

i) CREATE : Used to create a database or table or index.

Syntax:

*CREATE TABLE tableName
(column1 datatype1, column2 datatype2, ... columnN datatypeN);*

Example:

*CREATE TABLE student(rno int, name varchar(10), marks
decimal(7,2));*

ii) ALTER : Used to add a new column, remove existing column, modify the column, change the name of the column.

Syntax-1: To add a new column

ALTER TABLE tableName ADD columnName datatype;

Example-1:

ALTER TABLE student ADD branch varchar(10);

Syntax-2: To remove existing column

ALTER TABLE tableName DROP columnName;

Example-2:

ALTER TABLE student DROP branch;

Syntax-3: To modify datatype of the column

ALTER TABLE tableName MODIFY columnName newDatatype;

Example-3:

ALTER TABLE student MODIFY marks int;

Syntax-4: To change the name of the column

*ALTER TABLE tableName CHANGE oldColumnName
newColumnName datatype;*

Example-4:

ALTER TABLE student CHANGE name sname varchar(10);

iii) TRUNCATE : Used to clear the data inside the table and makes the table empty.

Syntax:

TRUNCATE tableName;

Example:

TRUNCATE student;

iv) DROP: Used to delete the table and also removes all the data in the table.

Syntax:

DROP table tableName;

Example:

DROP table student;

v) RENAME: Used to Rename the table.

Syntax-1:

RENAME <oldTableName> TO <NewTableName>;

Example:

RENAME student TO studentdemo;

Syntax-2:

ALTER TABLE RENAME <oldTableName> TO <NewTableName>;

Example:

ALTER TABLE RENAME student TO studentdemo;

4. WORKING WITH DML COMMANDS, WHERE CLAUSE

DML Commands

i) INSERT INTO :- Used to insert a new record(s) in a table.

Syntax-1:

*INSERT INTO tableName(col1,col2,...colN) VALUES
(value1,value2,...,valueN);*

Syntax-2: *INSERT INTO tableName VALUES (value1,value2,...,valueN);*

Syntax-3: *INSERT INTO tableName VALUES
(value1,value2,...,valueN),(value1,value2,...,valueN),
...,(value1,value2,...,valueN);*

Examples:

a) Below query insert single record in student table.

INSERT INTO student(rno,name,marks) VALUES (1, 'smith',60);

b) Below query insert single record in student table.

INSERT INTO student VALUES(2, 'john',70);

c) Below query insert 3 records in student table.

INSERT INTO student VALUES(3, 'clark',75),(4, 'Ram',80),(5, 'james',90);

ii) SELECT :- Used to fetch record(s) from a table.

Syntax-1:

SELECT col1,col2,...,colN FROM tableName;

Syntax-2:

*SELECT * FROM tableName;*

Examples:

a) Below query display student roll and name.

SELECT rno,name FROM student;

b) Below query display all details of a student.

*SELECT * FROM student;*

```
mysql> select * from student;
+-----+-----+-----+
| rno  | name  | marks |
+-----+-----+-----+
| 1    | smith | 60.00 |
| 2    | john  | 70.00 |
| 3    | clark | 75.00 |
| 4    | Ram   | 80.00 |
| 5    | james | 90.00 |
+-----+-----+-----+
```

iii) UPDATE :- Used to modify record(s) in a table.

Syntax

UPDATE tableName SET colName=value [WHERE condition];

Example

Below query updates the marks of smith.

UPDATE student SET marks=65 WHERE name='smith';

iv) DELETE :- Used to remove record(s) from a table.

Syntax:

DELETE FROM tableName [WHERE condition];

Example:

Below query deletes john record.

DELETE FROM student WHERE name='john';

WHERE clause

- Used with a SELECT statement to fetch specific records from the table.
- It is followed by a condition which could return true to fetch specific records.
- Used with UPDATE and DELETE statements to modify and remove specific records from the table when a given condition is true.

Syntax-1

SELECT col1,col2,...colN FROM tableName WHERE condition;

Syntax-2

UPDATE tableName SET columnName=value WHERE condition;

Syntax-3

DELETE FROM tableName WHERE condition;

Examples:

- a) Query to fetch 'smith' record
*SELECT * FROM student WHERE name='smith';*
- b) Query to update marks of 'Ram' record
UPDATE student SET marks=85 WHERE name='Ram';
- c) Query to delete 'john' record
DELETE FROM student WHERE name='john';
- d) Query to display student between roll numbers 2 and 5;
*SELECT * FROM student WHERE rno>2 AND rno<5;*
- e) Query to display both 'smith' and 'Ram' records.
*SELECT * FROM student WHERE name='smith' OR name='Ram';*

5. WORKING WITH INTEGRITY CONSTRAINTS

Integrity Constraints

- These are some rules to impose on data in the table for restricting or inserting irrelevant data in the tables.
- Used for data validation and verification.

Below are the integrity constraints:

- a. default** – Allows to change the default value for a column in table
- b. not null** – Allows restriction to store NULL values for a column in a table
- c. unique** – Allows to store only Unique values for a column in a table. No duplicate values are allowed on a column.
- d. check** – Allows restriction to provide a range of values for a column in a table.
- e. primary key**
- f. foreign key**

Examples:

- a. Applying default constraint
ALTER TABLE student MODIFY marks decimal(5,2) DEFAULT 0;
- b. Applying not null constraint
ALTER TABLE student MODIFY marks decimal(5,2) NOT NULL;
- c. Applying unique constraint
ALTER TABLE student MODIFY rno int UNIQUE;
- d. Applying check constraint
ALTER TABLE student ADD CONSTRAINT chk1 CHECK(marks>10 and marks<=100);

What is PRIMARY KEY?

- It is a special column that stores UNIQUE values to identify each row uniquely in a table.
- It doesn't store NULL values.
- It is always recommended to have a PRIMARY KEY column in a table.

Note:

PRIMARY KEY = UNIQUE+NOT NULL

Syntax-1: During Table Creation (using CREATE)

CREATE TABLE <table_name1> (Col1 DataType PRIMARY KEY, Col2,...ColN);

Example:

CREATE TABLE student(rno int PRIMARY KEY,name varchar(10));

Syntax-2: After Table Creation (using ALTER)

ALTER TABLE <table_name1> MODIFY Col1 DataType PRIMARY KEY;

Example:

ALTER TABLE student MODIFY rno int PRIMARY KEY;

What is Foreign KEY?

- It is a Column in One Table that refers to the PRIMARY KEY Column in another Table.
- It is used to link two tables in a database.

Syntax-1: During Table Creation (using CREATE)

*CREATE TABLE <table_name2>
(Col1 DataType, Col2,...ColN, FOREIGN KEY(Column-x) REFERENCES
<table_name1>(Column-x));*

Syntax-2: After Table Creation (using ALTER)

*ALTER TABLE <child_table_name>
ADD CONSTRAINT <constraint_name>
FOREIGN KEY(column_name)
REFERENCES <parent_table_name>(column_name);*

Note:

- A row in Parent Table cannot be deleted which was Referencing in Child Table
- A row cannot be inserted in the Child Table which was not Referenced in the Parent Table.

Query-1: Write a query to display student rollnos,names and their enrolled courses.

Solution:

SELECT rno,name,cname FROM student s,course c WHERE s.cid=c.cid;

Query-2: Write a query to display student rollnos,names and their enrolled course name whose credits is greater than 2.

Solution:

```
SELECT rno,name,cname FROM student s,course c WHERE s.cid=c.cid AND  
c.credits>2;
```

AUTO_INCREMENT

- It is a special feature used to insert a unique value Automatically for Primary Key attribute when a new Record is inserted.
- By default, auto_increment value begins from 1 and increments by 1 for each record insertion.
- If auto_increment is not set, by default it sets to 0 and does not increment further.
- We can change step size using auto_increment_increment.

Note-1: Setting the AUTO_INCREMENT

Example-1: Creating a student table ,inserting and displaying 3 rows.

```
CREATE TABLE student(rno int PRIMARY KEY AUTO_INCREMENT, name varchar(10));  
INSERT INTO student(name) VALUES('abc'),('xyz'),('smith');  
SELECT * FROM student;
```

rno	name
1	abc
2	xyz
3	smith

Example-2: Setting Auto_Increment using Alter command

```
ALTER TABLE student MODIFY rno int AUTO_INCREMENT
```

Note-2: Setting the step size of AUTO_INCREMENT

Example-1: Setting step size value to 5

```
SET @@AUTO_INCREMENT_INCREMENT=5;
```

Note-3: Removing AUTO_INCREMENT value for PRIMARY KEY

Example-1:

```
ALTER TABLE student MODIFY rno int;
```

6. BUILT-IN FUNCTIONS: MATH, DATE FUNCTIONS

A) Math functions

Math function	Purpose	Example
ceil(x)	Generates nearest highest value of x	Select ceil(2.3);
floor(x)	Generates nearest lowest value of x	Select floor(2.7);
round(x)	Generates nearest value of x	Select round(2.6);
rand()	Generates a random number between 0 to 1	Select rand();
PI()	Produces a pi value 3.14	Select pi();
sqrt(x)	Produces square root of x	Select sqrt(25);
pow(x,y)	Produces x power of y	Select pow(2,3);

Example Query: Write a query to generate a random number from 0 to 100

ANSWER:

*SELECT round(rand()*100);*

Example Query: Write a query to generate a random number from 50 to 100

ANSWER:

*SELECT round(50+rand()*50);*

B) Date functions

Date function	Purpose	Example
now()	Displays current date and time	Select now();
date(now())	Displays current date	Select date(now());
time(now())	Displays current time	Select time(now());
curdate()	Displays current date	Select curdate();

curtime()	Displays current time	Select curtime();
date_format()	Displays date and time in a specific format	Select date_format(curdate(), '%d-%m-%Y');
datediff()	Displays no. of days between current date and previous date	Select datediff(curdate(), '2013-04-23');
date_add()	Displays incremental day/month/year	Select date_add(curdate(), interval 1 day);
date_sub()	Displays decremental day/month/year	Select date_sub(curdate(), interval 1 day);

Example Query: Write a query to display age of a person

Answer: Assume Date of birth of a person is 12-08-2012

SELECT round(datediff(curdate(), '2012-08-12')/365) as age;

```
+-----+
| age   |
+-----+
|    12 |
+-----+
```

Example Query: Write a query to display date and time in following format:

12-09-2024,Thursday,10:05:23 AM

Answer:

SELECT date_format('2024-09-12', '%d-%m-%Y,%W,%r') as Result;

```
+-----+
| Result
+-----+
| 12-09-2024,Thursday,12:00:00 AM |
+-----+
```

7. Usage of Operators: BETWEEN, LIKE, IS NULL, LIMIT, DISTINCT, IN, NOT IN, CASE

To work with the above topics, we consider the student table with following data.

```
MariaDB [22oct24]> SELECT * FROM student;
```

rno	name	marks	gender	branch
1	smith	50	male	cse
2	ram	65	male	ece
3	sita	70	female	ece
4	mary	70	female	cse
5	allen	45	male	eee
6	clark	85	male	ece
7	rachel	90	female	cse

I) BETWEEN :- Used to fetch records within a range of values.

Example: To retrieve students between 2 and 5

*SELECT * FROM student WHERE rno BETWEEN 2 and 5;*

rno	name	marks	gender	branch
2	ram	65	male	ece
3	sita	70	female	ece
4	mary	70	female	cse
5	allen	45	male	eee

II) LIKE clause :- Used to fetch records that match with the given string pattern.

Two patterns are used.

% :- represents multiple characters match

Underscore(_) :- represents single character match

Example Queries:

Query	Output															
<i>SELECT * FROM student WHERE name LIKE 's%';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>1</td><td>smith</td><td>50</td><td>male</td><td>cse</td></tr><tr><td>3</td><td>sita</td><td>70</td><td>female</td><td>ece</td></tr></table>	rno	name	marks	gender	branch	1	smith	50	male	cse	3	sita	70	female	ece
rno	name	marks	gender	branch												
1	smith	50	male	cse												
3	sita	70	female	ece												
<i>SELECT * FROM student WHERE name LIKE '%m';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>2</td><td>ram</td><td>65</td><td>male</td><td>ece</td></tr></table>	rno	name	marks	gender	branch	2	ram	65	male	ece					
rno	name	marks	gender	branch												
2	ram	65	male	ece												
<i>SELECT * FROM student WHERE name LIKE 's%h';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>1</td><td>smith</td><td>50</td><td>male</td><td>cse</td></tr></table>	rno	name	marks	gender	branch	1	smith	50	male	cse					
rno	name	marks	gender	branch												
1	smith	50	male	cse												
<i>SELECT * FROM student WHERE name LIKE '%E%';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>5</td><td>allen</td><td>45</td><td>male</td><td>eee</td></tr><tr><td>7</td><td>rachel</td><td>90</td><td>female</td><td>cse</td></tr></table>	rno	name	marks	gender	branch	5	allen	45	male	eee	7	rachel	90	female	cse
rno	name	marks	gender	branch												
5	allen	45	male	eee												
7	rachel	90	female	cse												
<i>SELECT * FROM student WHERE name LIKE 'M__';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>4</td><td>mary</td><td>70</td><td>female</td><td>cse</td></tr></table>	rno	name	marks	gender	branch	4	mary	70	female	cse					
rno	name	marks	gender	branch												
4	mary	70	female	cse												
<i>SELECT * FROM student WHERE name LIKE '____';</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>3</td><td>sita</td><td>70</td><td>female</td><td>ece</td></tr><tr><td>4</td><td>mary</td><td>70</td><td>female</td><td>cse</td></tr></table>	rno	name	marks	gender	branch	3	sita	70	female	ece	4	mary	70	female	cse
rno	name	marks	gender	branch												
3	sita	70	female	ece												
4	mary	70	female	cse												

<i>SELECT * FROM student WHERE name LIKE ‘_l%’;</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>5</td><td>allen</td><td>45</td><td>male</td><td>eee</td></tr><tr><td>6</td><td>clark</td><td>85</td><td>male</td><td>ece</td></tr></table>	rno	name	marks	gender	branch	5	allen	45	male	eee	6	clark	85	male	ece
rno	name	marks	gender	branch												
5	allen	45	male	eee												
6	clark	85	male	ece												
<i>SELECT * FROM student WHERE name LIKE ‘__m’;</i>	<table><tr><th>rno</th><th>name</th><th>marks</th><th>gender</th><th>branch</th></tr><tr><td>2</td><td>ram</td><td>65</td><td>male</td><td>ece</td></tr></table>	rno	name	marks	gender	branch	2	ram	65	male	ece					
rno	name	marks	gender	branch												
2	ram	65	male	ece												

III) LIMIT clause :- To fetch first or last N records.

For example, to fetch first 3 records, we use

*SELECT * FROM student LIMIT 3;*

IV) IS NULL :- used to check for Null records

*SELECT * FROM student WHERE name IS NULL;*

V) IS NOT NULL :- used to check for Non-Null records

*SELECT * FROM student WHERE name IS NOT NULL;*

VI) DISTINCT :- Used to display unique tuples of a column

```
MariaDB [22oct24]> SELECT branch FROM student;
```

branch
cse
ece
ece
cse
eee
ece
cse

```
7 rows in set (0.001 sec)
```

```
MariaDB [22oct24]> SELECT DISTINCT branch FROM student;
```

branch
cse
ece
eee

```
3 rows in set (0.000 sec)
```

VII) IN : Used to fetch records by looking into a set of entries. A record is fetched if the given condition is matched with the set of entries.

```
MariaDB [22oct24]> SELECT * FROM student WHERE marks IN (45,90,65);
```

rno	name	marks	gender	branch
2	ram	65	male	ece
5	allen	45	male	eee
7	rachel	90	femal	cse

```
3 rows in set (0.001 sec)
```

VIII) NOT IN: Feches records which are not in the set of entries.

```
MariaDB [22oct24]> SELECT * FROM student WHERE marks NOT IN (45,90,65);
```

rno	name	marks	gender	branch
1	smith	50	male	cse
3	sita	70	femal	ece
4	mary	70	femal	cse
6	clark	85	male	ece

```
4 rows in set (0.001 sec)
```

IX) CASE :-

- The CASE statement goes through a set of conditions and returns a value when the first condition is met (like an IF-THEN-ELSE statement). So, once a condition is true, it will stop reading and return the result.
- If no conditions are true, it will return the value in the ELSE clause.
- If there is no ELSE part and no conditions are true, it returns NULL

SYNTAX

CASE

WHEN condition1 THEN result1

WHEN condition2 THEN result2

WHEN conditionN THEN resultN

ELSE result

END;

Example:

Below Query display status of students. Display “PASS” for those marks >70, “PROMOTED” for those between 40 and 69, and “FAIL” for those marks below 40.

```
SELECT rno,name,  
CASE  
WHEN marks >=70 THEN "PASS"  
WHEN marks BETWEEN 40 and 69 THEN  
"PROMOTED"  
ELSE "FAIL"  
END AS status  
FROM student;
```

rno	name	status
1	smith	PROMOTED
2	ram	PROMOTED
3	sita	PASS
4	mary	PASS
5	allen	PROMOTED
6	clark	PASS
7	rachel	PASS

8. WORKING WITH SET OPERATORS AND JOINS

The following are the SET Operators

- a) UNION
- b) INTERSECT
- c) EXCEPT
- d) CARTESIAN PRODUCT

Note-1: To demonstrate this, we created TWO tables: **student2** and **emp2** as shown below:

```
CREATE TABLE student2(rno int,name varchar(10),marks int);  
CREATE TABLE emp2(eid int,name varchar(10),salary int);
```

Note-2: We inserted a few records in both **student2** and **emp2** tables.

```
INSERT INTO student2 VALUES(1,'smith',50);  
INSERT INTO student2 VALUES(2,'sita',60);  
INSERT INTO student2 VALUES(3,'john',70),(4,'james',70);  
INSERT INTO student2 VALUES(5,'allen',76);
```

```
INSERT INTO emp2 VALUES(10,'ram',3000);  
INSERT INTO emp2 VALUES(2,'sita',40);  
INSERT INTO emp2 VALUES(30,'john',7000),(4,'shyam',70);  
INSERT INTO emp2 VALUES(5,'allen',76);
```

Note-3: We displayed both tables

MariaDB [cse999]> select * from student2;

rno	name	marks
1	smith	50
2	sita	60
3	john	70
4	james	70
5	allen	76

5 rows in set (0.000 sec)

MariaDB [cse999]> select * from emp2;

eid	name	salary
10	ram	3000
2	sita	40
30	john	7000
4	shyam	70
5	allen	76

5 rows in set (0.000 sec)

UNION

- It is used to merge records from the given two tables.
- Removes Duplicate records
- Number of columns and types should be the same between two tables.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 UNION SELECT * FROM emp2;
```

rno	name	marks
1	smith	50
2	sita	60
3	john	70
4	james	70
5	allen	76
10	ram	3000
2	sita	40
30	john	7000
4	shyam	70

```
9 rows in set (0.001 sec)
```

UNION ALL

- It is also used to merge records from the given two tables.
- Includes Duplicate records

Example:

```
MariaDB [cse999]> SELECT * FROM student2 UNION ALL SELECT * FROM emp2;
```

rno	name	marks
1	smith	50
2	sita	60
3	john	70
4	james	70
5	allen	76
10	ram	3000
2	sita	40
30	john	7000
4	shyam	70
5	allen	76

```
10 rows in set (0.000 sec)
```

INTERSECT

- Displays common records from two tables.
- Number of columns and types should be the same between two tables.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 INTERSECT SELECT * FROM emp2;
```

rno	name	marks
5	allen	76

1 row in set (0.000 sec)

EXCEPT

- It is a Set Difference operation
- Displays records from table-1 which do not exist in table-2.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 EXCEPT SELECT * FROM emp2;
```

rno	name	marks
1	smith	50
2	sita	60
3	john	70
4	james	70

4 rows in set (0.000 sec)

CARTESIAN PRODUCT

If Table-1 has M rows and Table-2 has N rows, then cartesian product merges tuples from both tables and generates M*N rows. There may exist redundant tuples.

Example: A student table has 2 rows and an emp table has 3 rows.

SELECT * from student,emp;

rno	name	marks	eid	ename
1	smith	60.00	1	john
2	john	70.00	1	john
1	smith	60.00	2	Allen
2	john	70.00	2	Allen
1	smith	60.00	3	ward
2	john	70.00	3	ward

JOINS

They are used to combine two or more tables using a condition or common attribute between tables.

Types of Joins

- a) **INNER JOIN :-** Combines two tables T1,T2 using a given condition and displays all matched records in T1,T2.

Example-1: (EQUI-JOIN)

```
MariaDB [cse999]> SELECT * FROM student2 INNER JOIN emp2 ON student2.name = emp2.name;
```

rno	name	marks	eid	name	salary
2	sita	60	2	sita	40
3	john	70	30	john	7000
5	allen	76	5	allen	76

3 rows in set (0.001 sec)

Example-2: (THETA-JOIN)

```
MariaDB [cse999]> SELECT * FROM student2 INNER JOIN emp2 ON student2.rno > emp2.eid;
```

rno	name	marks	eid	name	salary
3	john	70	2	sita	40
4	james	70	2	sita	40
5	allen	76	2	sita	40
5	allen	76	4	shyam	70

4 rows in set (0.001 sec)

- b) **NATURAL JOIN :-** Combines two tables T1,T2 using a common attribute in T1,T2 and displays all matched records from T1,T2.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 NATURAL JOIN emp2;
```

name	rno	marks	eid	salary
sita	2	60	2	40
john	3	70	30	7000
allen	5	76	5	76

3 rows in set (0.001 sec)

- c) **LEFT OUTER JOIN :-** Combines Two tables T1,T2 and displays all records from T1 and matched & unmatched records of T2. Unmatched records are indicated with NULL.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 LEFT OUTER JOIN emp2 ON student2.name=emp2.name;
```

rno	name	marks	eid	name	salary
2	sita	60	2	sita	40
3	john	70	30	john	7000
5	allen	76	5	allen	76
1	smith	50	NULL	NULL	NULL
4	james	70	NULL	NULL	NULL

5 rows in set (0.000 sec)

- d) **RIGHT OUTER JOIN:-** Combines Two tables T1,T2 and displays all records from T2 and matched & unmatched records of T1. Unmatched records are indicated with NULL.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 RIGHT OUTER JOIN emp2 ON student2.name=emp2.name;
```

rno	name	marks	eid	name	salary
2	sita	60	2	sita	40
3	john	70	30	john	7000
5	allen	76	5	allen	76
NULL	NULL	NULL	10	ram	3000
NULL	NULL	NULL	4	shyam	70

5 rows in set (0.000 sec)

- e) **FULL OUTER JOIN:-** Combines Two tables T1,T2 and displays all matched & unmatched records of T1 & T2. Unmatched records are indicated with NULL.

Note: In MYSQL, we combine LEFT,RIGHT OUTER Joins using UNION to achieve FULL OUTER JOIN.

Example:

```
MariaDB [cse999]> SELECT * FROM student2 LEFT OUTER JOIN emp2 ON student2.name=emp2.name  
→ UNION  
→ SELECT * FROM student2 RIGHT OUTER JOIN emp2 ON student2.name=emp2.name;
```

rno	name	marks	eid	name	salary
2	sita	60	2	sita	40
3	john	70	30	john	7000
5	allen	76	5	allen	76
1	smith	50	NULL	NULL	NULL
4	james	70	NULL	NULL	NULL
NULL	NULL	NULL	10	ram	3000
NULL	NULL	NULL	4	shyam	70

7 rows in set (0.002 sec)

9. WORKING WITH VIEWS

VIEW: A VIEW is a virtual table generated from the resultant query of another table.

- It is a derived table.
- If any changes are done in the original table, that also reflects in VIEW and vice versa.

Syntax for creating VIEW:

```
CREATE VIEW viewName  
AS  
SELECT column1,column2,...columnN  
FROM tableName [WHERE condition];
```

Syntax for Displaying VIEW :

```
SELECT * FROM viewName;
```

Syntax for Dropping VIEW:

```
DROP VIEW viewName;
```

Examples:

Note: Consider the student table with following data.

rno	name	marks	gender	branch
1	smith	50	male	cse
2	ram	65	male	ece
3	sita	70	female	ece
4	mary	70	female	cse
5	allen	45	male	eee
6	clark	85	male	ece
7	rachel	90	female	cse

- 1) Create a view to display only cse records.

```
CREATE VIEW cseview AS SELECT * FROM student WHERE  
branch='cse';
```

```
MariaDB [22oct24]> CREATE VIEW cseview AS  
-> SELECT * FROM student WHERE branch='cse';  
Query OK, 0 rows affected (0.117 sec)
```

```
MariaDB [22oct24]> SELECT * FROM cseview;
```

rno	name	marks	gender	branch
1	smith	75	male	cse
4	mary	70	female	cse
7	rachel	90	female	cse

- 2) Create a view to display only female records.

```
CREATE VIEW femaleview AS SELECT * FROM student WHERE  
gender='female';
```

```
MariaDB [22oct24]> SELECT * FROM femaleview;
```

rno	name	marks	gender	branch
3	sita	70	female	ece
4	mary	70	female	cse
7	rachel	90	female	cse

- Updating marks of 'smith' from 70 to 75 in cseview. It updates in the original table.

```
UPDATE cseview SET marks=75 WHERE rno=1;
```

```
MariaDB [22oct24]> SELECT * FROM student;
```

rno	name	marks	gender	branch
1	smith	75	male	cse
2	ram	65	male	ece
3	sita	70	female	ece
4	mary	70	female	cse
5	allen	45	male	eee
6	clark	85	male	ece
7	rachel	90	female	cse

- We cannot add any new columns in the view. Will throw an error.

```
MariaDB [22oct24]> ALTER TABLE cseview ADD dob date;
ERROR 1347 (HY000): '22oct24.cseview' is not of type 'BASE TABLE'
```

→ Display existing views using the below query.

Show tables;

Tables_in_22oct24
cseview
femaleview
student

→ Below query will Delete a view:

DROP view cseview;

10. USAGE OF ORDER BY, GROUP BY, HAVING AND AGGREGATE FUNCTIONS

ORDER BY clause: Used to arrange the data in sorted form based on the specified attribute.

Syntax:

```
SELECT column1,column2,...columnN
FROM tableName [WHERE condition]
ORDER BY columnX
```

Note: Consider the employee table with following data.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10

Example Queries:

Query-1: Display employee names in ascending order.

```
SELECT ename FROM emp ORDER BY ename;
```

ename
ADAMS
ALLEN
BLAKE
CLARK
FORD
JAMES
JONES
KING
MARTIN
MILLER
SCOTT
SMITH
TURNER
WARD

Query-2: Display employee names in descending order.

SELECT ename FROM emp ORDER BY ename DESC;

ename
WARD
TURNER
SMITH
SCOTT
MILLER
MARTIN
KING
JONES
JAMES
FORD
CLARK
BLAKE
ALLEN
ADAMS

Query-3: Display departments in ascending order and employee names in descending order.

```
MariaDB [empdemo11]> SELECT deptno,ename FROM emp ORDER BY deptno,ename DESC;
```

deptno	ename
10	MILLER
10	KING
10	CLARK
20	SMITH
20	SCOTT
20	JONES
20	FORD
20	ADAMS
30	WARD
30	TURNER
30	MARTIN
30	JAMES
30	BLAKE
30	ALLEN

GROUP BY Clause

- It is used to collect data from multiple records and group the result by one or more columns.
- It is generally used in a SELECT statement and associates with aggregate functions.

Syntax

SELECT col1,col2,...colN,aggreagteFunction(column)

FROM tableName

WHERE condition GROUP BY col1,col2,..colN;

Note: To work with GROUP BY clause we used the following table and data.

MariaDB [empdemo11]> SELECT * FROM emp;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10

14 rows in set (0.001 sec)

Example Queries:

Query-1: Display number of employees in each dept.

MariaDB [empdemo11]> SELECT deptno,count(*) FROM emp GROUP BY deptno;

deptno	count(*)
10	3
20	5
30	6

3 rows in set (0.005 sec)

Query-2: Display number of employees in each job.

MariaDB [empdemo11]> SELECT job,count(*) FROM emp GROUP BY job;

job	count(*)
ANALYST	2
CLERK	4
MANAGER	3
PRESIDENT	1
SALESMAN	4

5 rows in set (0.000 sec)

Query-3: Display maximum, minimum, total and average salary in each dept.

```
MariaDB [empdemo11]> SELECT deptno,max(sal),min(sal),avg(sal) FROM emp GROUP BY deptno;
```

deptno	max(sal)	min(sal)	avg(sal)
10	5000.00	1300.00	2916.666667
20	3000.00	800.00	2175.000000
30	2850.00	950.00	1566.666667

3 rows in set (0.001 sec)

Query-4: Display maximum, minimum,total and average salary in each job.

```
MariaDB [empdemo11]> SELECT job,max(sal),min(sal),avg(sal) FROM emp GROUP BY job;
```

job	max(sal)	min(sal)	avg(sal)
ANALYST	3000.00	3000.00	3000.000000
CLERK	1300.00	800.00	1037.500000
MANAGER	2975.00	2450.00	2758.333333
PRESIDENT	5000.00	5000.00	5000.000000
SALESMAN	1600.00	1250.00	1400.000000

5 rows in set (0.001 sec)

HAVING clause

- It is used to filter the data among grouped records.
- It filters the grouped records by applying conditions on aggregate functions.
- Used with GROUP BY clauses.

SYNTAX

*SELECT col1,col2,...colN,aggreateFunction(column) FROM tableName
WHERE condition GROUP BY col1,col2,...colN HAVING condition;*

Example Queries

Query-1: Display number of depts that contains more than 3 employees.

```
mysql> SELECT deptno,count(*) from EMP GROUP BY deptno HAVING count(*)>3;
+-----+
| deptno | count(*) |
+-----+
| 20     | 5        |
| 30     | 6        |
+-----+
2 rows in set (0.01 sec)
```

Query-2: Display number of jobs whose employees are less than 4;

```
mysql> SELECT job,count(*) from EMP GROUP BY job HAVING count(*)<4;
+-----+-----+
| job      | count(*) |
+-----+-----+
| ANALYST  |         2 |
| MANAGER  |         3 |
| PRESIDENT|         1 |
+-----+-----+
3 rows in set (0.00 sec)
```

Query-3: Display depts who have a maximum salary < 3000.

```
mysql> SELECT deptno,max(sal) from EMP GROUP BY deptno HAVING max(sal)<3000;
+-----+-----+
| deptno | max(sal) |
+-----+-----+
|      30 | 2850.00 |
+-----+-----+
1 row in set (0.00 sec)
```

Query-4: Display the jobs whose maximum salary is greater than 2000.

```
mysql> SELECT job,max(sal) from EMP GROUP BY job HAVING max(sal)>2000;
+-----+-----+
| job      | max(sal) |
+-----+-----+
| ANALYST  | 3000.00 |
| MANAGER  | 2975.00 |
| PRESIDENT| 5000.00 |
+-----+-----+
3 rows in set (0.00 sec)
```

Query-5: Display depts who have an average salary from 2000 to 3000.

```
mysql> SELECT deptno,avg(sal) from EMP GROUP BY deptno
-> HAVING avg(sal) BETWEEN 2000 AND 3000;
+-----+-----+
| deptno | avg(sal) |
+-----+-----+
|      10 | 2916.666667 |
|      20 | 2175.000000 |
+-----+-----+
2 rows in set (0.00 sec)
```

Difference between WHERE and HAVING clause

WHERE : It is used to filter the data by applying conditions on individual rows.

We cannot use aggregate functions as condition

HAVING : It is used to filter the grouped data by applying conditions on grouped rows. We can use aggregate functions as conditions.

Order of Keywords

SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT

AGGREGATE Functions :

- a) count(*) , count(AttributeName) :- count the number of tuples for an attribute
- b) sum(AttributeName) :- returns the sum of tuples for an attribute
- c) min(AttributeName) :- returns the maximum value in the tuples of an attribute
- d) max(AttributeName) :- returns the minimum value in the tuples of an attribute
- e) avg(AttributeName) :- returns the average value of the tuples of an attribute

Examples Queries:

Query-1: Display number of tuples in the employee table.

```
MariaDB [empdemo11]> select count(*) from emp;
+-----+
| count(*) |
+-----+
|         14 |
+-----+
1 row in set (0.002 sec)
```

Query-2: Display the number of commission values in the employee table.

```
MariaDB [empdemo11]> select count(comm) from emp;
+-----+
| count(comm) |
+-----+
|           4 |
+-----+
1 row in set (0.000 sec)
```

Query-3: Display the sum,maximum,minimum and average values in the employee table

```
MariaDB [empdemo11]> select sum(sal),max(sal),min(sal),avg(sal) from emp;
+-----+-----+-----+-----+
| sum(sal) | max(sal) | min(sal) | avg(sal) |
+-----+-----+-----+-----+
| 29025.00 | 5000.00 | 800.00 | 2073.214286 |
+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

Query-4: Display the number of unique departments in the employee table.

```
MariaDB [empdemo11]> select count(distinct deptno) from emp;
+-----+
| count(distinct deptno) |
+-----+
| 3 |
+-----+
1 row in set (0.001 sec)
```

11. WORKING WITH SUBQUERIES

SubQuery:

It is a query within another query.

- A query which is nested in another SQL query embedded with a SELECT statement.
- Nested query is the Inner Query and Main query will be Outer Query.
- Nested Query executes First and its result will be given to Outer Query.

Syntax

SELECT column(s) FROM tableName

WHERE column OPERATOR

(SELECT column(s) FROM tableName [where]);

Note: To demonstrate subqueries, we consider the employee and department tables with following data.

MariaDB [empdemo11]> SELECT * FROM emp;								
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20	
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30	
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30	
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20	
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30	
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30	
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10	
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20	
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10	
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30	
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20	
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30	
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20	
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10	

```
MariaDB [empdemo11]> SELECT * FROM dept;
```

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Example Queries

i) Using Relational Operators (>,<=,!=,>=,<=)

Query-1: Display names of employees who are having a maximum salary.

```
mysql> SELECT * FROM EMP WHERE sal = (SELECT max(sal) FROM EMP);
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10

1 row in set (0.00 sec)

Query-2: Display names of both maximum and minimum salaried employees.

```
mysql> SELECT * FROM EMP WHERE sal = (SELECT max(sal) FROM EMP)
-> OR
-> sal = (SELECT min(sal) FROM EMP);
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10

2 rows in set (0.00 sec)

Query-3: Display names of employees whose salary is greater than average salary.

```
mysql> SELECT * FROM EMP WHERE sal > (SELECT avg(sal) FROM EMP);
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20

```
6 rows in set (0.00 sec)
```

Query-4: Display empno,ename of employees whose manager id is highest.

```
mysql> SELECT empno,ename,mgr from EMP WHERE
-> mgr=(SELECT max(mgr) FROM EMP);
```

empno	ename	mgr
7369	SMITH	7902

```
1 row in set (0.00 sec)
```

Query-5: List the employees who are senior to King.

```
MariaDB [empdemo11]> SELECT * FROM emp WHERE hiredate
-> < (SELECT hiredate FROM emp WHERE ename='KING');
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30

Query-6: List the details of the employees whose Salaries are more than the employee BLAKE.

```
MariaDB [empdemo11]> SELECT * FROM emp WHERE sal >
→ (SELECT sal FROM emp WHERE ename='BLAKE');
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20

Query-7: List the employees who are not working in SALES dept.

```
MariaDB [empdemo11]> SELECT * FROM emp WHERE deptno ≠
→ (SELECT deptno FROM dept WHERE dname='SALES');
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10

2) Using IN,NOT IN Operator

Query-1: Display names of employees who are working as 'salesman'

```
mysql> SELECT ename,job FROM EMP WHERE job
-> IN
-> (SELECT job FROM EMP WHERE job='salesman');
+-----+-----+
| ename | job      |
+-----+-----+
| ALLEN | SALESMAN |
| WARD  | SALESMAN |
| MARTIN | SALESMAN |
| TURNER | SALESMAN |
+-----+-----+
4 rows in set (0.00 sec)
```

Query-2: Display names of employees who are not working as 'salesman' and 'clerk'

```
mysql> SELECT ename,job FROM EMP WHERE job
-> NOT IN
-> (SELECT job FROM EMP WHERE job='salesman' or job='clerk');
+-----+-----+
| ename | job      |
+-----+-----+
| JONES | MANAGER  |
| BLAKE | MANAGER  |
| CLARK | MANAGER  |
| SCOTT | ANALYST  |
| KING  | PRESIDENT|
| FORD  | ANALYST  |
+-----+-----+
6 rows in set (0.00 sec)
```

Query-3: Display the names of the emps who are getting the highest salary dept wise.

```
mysql> SELECT deptno,ename,sal FROM EMP WHERE
-> sal IN (SELECT max(sal) FROM EMP GROUP BY deptno);
+-----+-----+-----+
| deptno | ename | sal      |
+-----+-----+-----+
|      30 | BLAKE | 2850.00 |
|      20 | SCOTT | 3000.00 |
|      10 | KING  | 5000.00 |
|      20 | FORD  | 3000.00 |
+-----+-----+-----+
4 rows in set (0.00 sec)
```

Query-4: Display the Departments where there are no employees.

```
mysql> SELECT deptno FROM DEPT WHERE deptno NOT IN (SELECT deptno FROM EMP);
+-----+
| deptno |
+-----+
|      40 |
+-----+
1 row in set (0.01 sec)
```

3) Using FROM clause

Query-1: Display maximum,minimum salary and avg commision from employees table.

```
mysql> SELECT max(sal),min(sal),avg(comm)
-> FROM (SELECT sal,comm FROM EMP) AS temp;
+-----+-----+-----+
| max(sal) | min(sal) | avg(comm) |
+-----+-----+-----+
| 5000.00 | 800.00 | 550.000000 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

12. PL/SQL : PROCEDURES

1) Write a procedure to check if a given number is even or odd.

Step-1: Execute below code in mysql terminal.

Step-2: Run the below commands to get the output

DELIMITER ;

CALL evenodd(5);

Procedure code	Output
<pre>DELIMITER // CREATE PROCEDURE evenodd(IN k INT) BEGIN DECLARE result VARCHAR(10); IF (k%2=0) THEN SET result="even"; ELSE SET result="odd"; END IF; SELECT k,result; END //</pre>	<pre>MariaDB [test]> DELIMITER ; MariaDB [test]> CALL evenodd(5); +-----+-----+ k result +-----+-----+ 5 odd +-----+-----+ MariaDB [test]> CALL evenodd(10); +-----+-----+ k result +-----+-----+ 10 even +-----+-----+</pre>

2) Write a procedure to find the factorial of a given number.

Step-1: Execute below code in mysql terminal.

Step-2: Run the below commands to get the output

DELIMITER ;

CALL factorial(5);

Procedure code	Output
<pre>DELIMITER // CREATE PROCEDURE factorial(IN k INT) BEGIN DECLARE result,i INT DEFAULT 1; REPEAT SET result=result*i; SET i=i+1; UNTIL i>k END REPEAT; SELECT k,result; END //</pre>	<pre>MariaDB [test]> DELIMITER ; MariaDB [test]> CALL factorial(5); +-----+-----+ k result +-----+-----+ 5 120 +-----+-----+ MariaDB [test]> CALL factorial(6); +-----+-----+ k result +-----+-----+ 6 720 +-----+-----+</pre>

13.PL/SQL FUNCTIONS

1) Write a function to compute simple interest.

Step-1: Execute below code in mysql terminal.

Step-2: Run the below commands to get the output

DELIMITER ;

SELECT si(5);

Function code	Output
<pre>DELIMITER // CREATE FUNCTION si(p INT,t INT,r FLOAT) RETURNS FLOAT BEGIN DECLARE result FLOAT DEFAULT 0; SET result=(p*t*r)/100; RETURN result; END //</pre>	<pre>MariaDB [test]> DELIMITER ; MariaDB [test]> SELECT SI(5000,2,8.5); +-----+ SI(5000,2,8.5) +-----+ 850 +-----+ MariaDB [test]> SELECT si(7000,3,2.3); +-----+ si(7000,3,2.3) +-----+ 483 +-----+</pre>

2) Write a function that accepts roll number and displays the name of the student from a student table.

Step-1: Create a Student table with following data.

MariaDB [test]> SELECT * FROM student;

rno	name	marks
20	smith	70
50	john	75
124	suresh	80

Step-2: Execute below code in mysql terminal

Step-3: Run the below commands to get the output

DELIMITER ;

SELECT getname(20);

SELECT getname(50);

SELECT getname(200);

Function code	Output
<pre>DELIMITER // CREATE FUNCTION getname(k INT) RETURNS VARCHAR(20) BEGIN DECLARE temp VARCHAR(20); SELECT name INTO temp FROM student WHERE rno=k; RETURN temp; END //</pre>	<pre>MariaDB [test]> DELIMITER ; MariaDB [test]> SELECT getname(50); +-----+ getname(50) +-----+ john +-----+ 1 row in set (0.000 sec) MariaDB [test]> SELECT getname(20); +-----+ getname(20) +-----+ smith +-----+ 1 row in set (0.000 sec)</pre>

14. TRIGGERS

1) Create a trigger that may insert a tuple into Table5 when a tuple is inserted into Table 4. Specifically, the trigger checks whether the new tuple has a first value greater than 10 or not, and if so inserts the reverse tuple into Table5.

Step-1: Create two tables Table4 and Table5 with following commands.

CREATE TABLE Table4(a int,b char);

CREATE TABLE Table5(c char,d int);

Step-2: Execute below trigger code in mysql terminal

DELIMITER //

CREATE TRIGGER demo AFTER INSERT ON Table4 FOR EACH ROW

BEGIN

IF (new.a > 10) THEN

INSERT INTO Table5 VALUES(new.b,new.a);

END IF;

END //

Step-3: Run the below INSERT command

INSERT INTO Table4 VALUES(20, 'A'),(6, 'B'),(3, 'C'),(20, 'D');

OUTPUT:

```
MariaDB [test]> DELIMITER ;
MariaDB [test]> SELECT * FROM Table4;
Empty set (0.000 sec)
```

```
MariaDB [test]> SELECT * FROM Table5;
Empty set (0.000 sec)
```

```
MariaDB [test]> INSERT INTO Table4 VALUES(20,'A'),(6,'B'),(3,'C'),(40,'D');
Query OK, 4 rows affected (0.028 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
MariaDB [test]> SELECT * FROM Table4;
```

a	b
20	A
6	B
3	C
40	D

```
4 rows in set (0.000 sec)
```

```
MariaDB [test]> SELECT * FROM Table5;
```

c	d
A	20
D	40

2) Create a trigger that sets the total sum in the accumulator variable before inserting a new row in Table4. First value of each row is added to the accumulator variable.

```
MariaDB [test]> delimiter ;  
MariaDB [test]> SET @sum=0;  
Query OK, 0 rows affected (0.000 sec)
```

```
MariaDB [test]> DELIMITER //  
MariaDB [test]> CREATE TRIGGER demo2 BEFORE INSERT ON Table4 FOR EACH ROW  
    → SET @sum=@sum+NEW.a; //  
Query OK, 0 rows affected (0.054 sec)
```

```
MariaDB [test]> DELIMITER ;  
MariaDB [test]> INSERT INTO Table4 VALUES(30,'x'),(40,'y'),(60,'z');  
Query OK, 3 rows affected (0.038 sec)  
Records: 3 Duplicates: 0 Warnings: 0
```

```
MariaDB [test]> SELECT @sum as 'Total Sum';
```

130

15.CURSORS

Query: Find the names and their jobs who are working in the particular department.

Solution:

Consider the following employee data from the EMP table.

```
mysql> SELECT * FROM EMP;
```

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800.00	NULL	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600.00	300.00	30
7521	WARD	SALESMAN	7698	1981-02-22	1250.00	500.00	30
7566	JONES	MANAGER	7839	1981-04-02	2975.00	NULL	20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250.00	1400.00	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850.00	NULL	30
7782	CLARK	MANAGER	7839	1981-06-09	2450.00	NULL	10
7788	SCOTT	ANALYST	7566	1982-12-09	3000.00	NULL	20
7839	KING	PRESIDENT	NULL	1981-11-17	5000.00	NULL	10
7844	TURNER	SALESMAN	7698	1981-09-08	1500.00	0.00	30
7876	ADAMS	CLERK	7788	1983-01-12	1100.00	NULL	20
7900	JAMES	CLERK	7698	1981-12-03	950.00	NULL	30
7902	FORD	ANALYST	7566	1981-12-03	3000.00	NULL	20
7934	MILLER	CLERK	7782	1982-01-23	1300.00	NULL	10

Step-1: Type following Cursor code in MySql terminal

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE empcursor(IN deptid INT)
-> BEGIN
-> DECLARE flag INTEGER DEFAULT 0;
-> DECLARE namee VARCHAR(20);
-> DECLARE jobe VARCHAR(10);
-> DECLARE c CURSOR FOR
-> SELECT ename,job FROM EMP WHERE deptno = deptid;
-> DECLARE CONTINUE HANDLER FOR NOT FOUND SET flag = 1;
-> OPEN c;
-> eloop: LOOP
-> FETCH c INTO namee,jobe;
-> IF flag THEN
-> LEAVE eloop;
-> END IF;
-> SELECT namee,jobe;
-> END LOOP eloop;
-> CLOSE c;
-> END //
```

Query OK, 0 rows affected (0.00 sec)

Step-2: Execute the cursor using below command

DELIMITER ;
CALL empcursor(10);

OUTPUT:

```
mysql> DELIMITER ;
mysql> CALL empcursor(10);
+-----+-----+
| namee | jobe   |
+-----+-----+
| CLARK | MANAGER |
+-----+-----+
1 row in set (0.00 sec)

+-----+-----+
| namee | jobe   |
+-----+-----+
| KING  | PRESIDENT |
+-----+-----+
1 row in set (0.00 sec)

+-----+-----+
| namee | jobe   |
+-----+-----+
| MILLER | CLERK |
+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

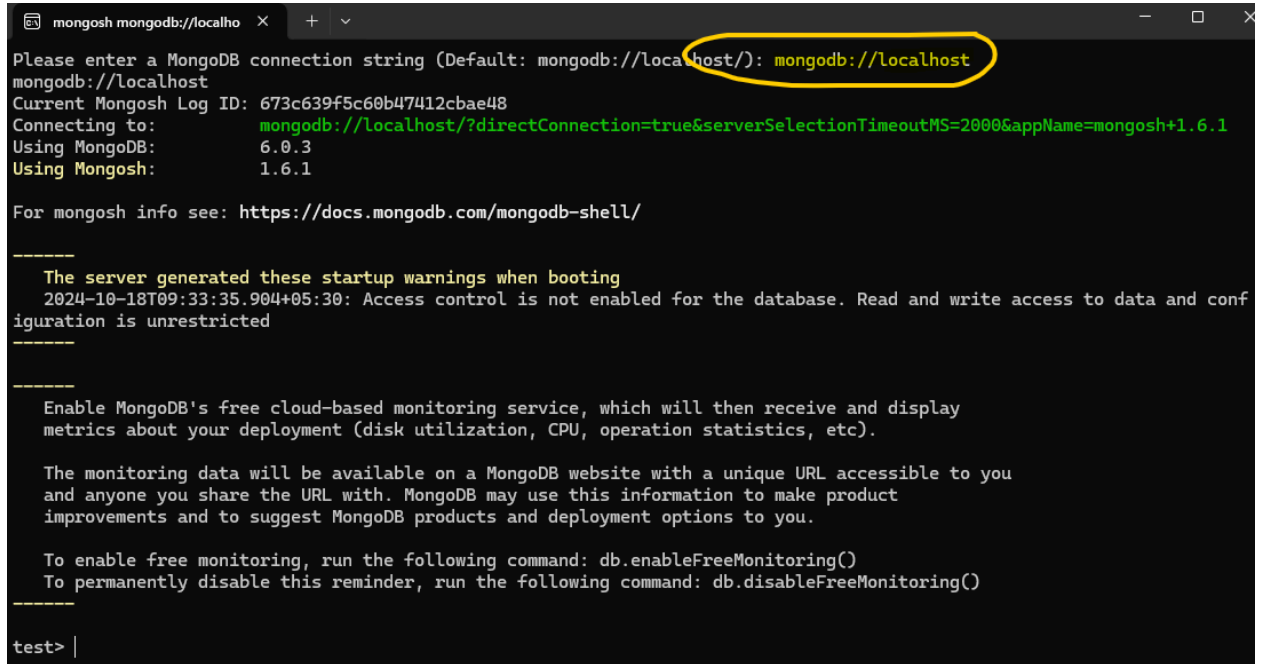
16.NoSQL:Basic CRUD operations using MongoDB

Steps to work with MongoDB

Step-1: Open Mongo Shell, type below command and press enter.

mongodb://localhost

Step-2: By default the mongo shell looks like the following.



```
mongosh mongodb://localhost x + v
Please enter a MongoDB connection string (Default: mongodb://localhost/): mongodb://localhost
mongodb://localhost
Current Mongosh Log ID: 673c639f5c60b47412cbae48
Connecting to: mongodb://localhost/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+1.6.1
Using MongoDB: 6.0.3
Using Mongosh: 1.6.1

For mongosh info see: https://docs.mongodb.com/mongosh-shell/

-----
The server generated these startup warnings when booting
2024-10-18T09:33:35.904+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

Enable MongoDB's free cloud-based monitoring service, which will then receive and display metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you and anyone you share the URL with. MongoDB may use this information to make product improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()

-----
test> |
```

Few Useful commands:

sl.no	Command	Description	Output
1	db	Display name of the current working database	<pre>test> db test</pre>
2	show databases	List out existing databases	<pre>test> show databases; admin 40.00 KiB config 96.00 KiB local 96.00 KiB test 72.00 KiB test></pre>
3	use mvsr	create / use new database named “mvsr”	<pre>test> use mvsr; switched to db mvsr mvsr></pre>

4	show tables	List out existing collections in current database	<pre> mvsr> show tables emp mvsr> </pre>
---	-------------	---	--

CRUD Operations using MongoDB

1) Creating and Inserting Employee Records

We use following methods to insert record(s)

a) **insertOne()** : Inserts a single record

Example-1: Inserting 'Smith' Record

```

db.emp.insertOne(
{
  '_id':110,
  'ename':'Smith',
  'age':33,
  'salary':7000
}
);

```

Example-2: Inserting 'John' Record

```

db.emp.insertOne(
{
  '_id':111,
  'ename':'John',
  'age':34,
  'salary':8000
}
);

```

```

mvsr> db.emp.insertOne({'_id':110,'ename':'Smith','age':33,'salary':7000});
{ acknowledged: true, insertedId: 110 }
mvsr> db.emp.insertOne({'_id':111,'ename':'John','age':34,'salary':8000});
{ acknowledged: true, insertedId: 111 }
mvsr>

```

b) **insertMany()**: Inserts multiple records

Example: Inserting 'James', 'Allen' records

```

db.emp.insertMany(
[
  {'_id':112,'ename':'James','age':35,'salary':7500},
  {'_id':113,'ename':'Allen','age':65,'salary':5600}
]
);

```

```

mvsr> db.emp.insertMany(
... [
...   {'_id':112,'ename':'James','age':35,'salary':7500},
...   {'_id':113,'ename':'Allen','age':65,'salary':5600}
... ]
... );
{ acknowledged: true, insertedIds: { '0': 112, '1': 113 } }
mvsr>

```

2) Fetching and Displaying Employee Records

We use following methods to display record(s)

- a) **find()** : display all records in the table

Example-1:

db.emp.find()

- b) **find({key:value})** : display a specified record with given key,value

Example-2:

db.emp.find({'_id':112})

```

mvsr> db.emp.find()
[
  { _id: 110, ename: 'Smith', age: 33, salary: 7000 },
  { _id: 111, ename: 'John', age: 34, salary: 8000 },
  { _id: 112, ename: 'James', age: 35, salary: 7500 },
  { _id: 113, ename: 'Allen', age: 65, salary: 5600 }
]
mvsr> db.emp.find({'_id':112})
[ { _id: 112, ename: 'James', age: 35, salary: 7500 } ]
mvsr>

```

3) Updating Employee Records

We use following methods to update record(s)

- a) **updateOne()** : updates a specific record with given key,value

Example-1 :Updating John's salary to 8500

db.emp.updateOne({'ename':'John'},{\$set: {'salary':8500}});

- b) **updateMany()** : updates all records in a table

Example-2: Updating emp's salary to 12000 whose age < 60

db.emp.updateMany({age:{\$lt:60}},{\$set: {'salary':8000}});

Output-1 (Before Update)

```
mvsr> db.emp.find()
[
  { _id: 110, ename: 'Smith', age: 33, salary: 7000 },
  { _id: 111, ename: 'John', age: 34, salary: 8000 },
  { _id: 112, ename: 'James', age: 35, salary: 7500 },
  { _id: 113, ename: 'Allen', age: 65, salary: 5600 }
]
```

Output-2 (After Update)

```
mvsr> db.emp.find()
[
  { _id: 110, ename: 'Smith', age: 33, salary: 8000 },
  { _id: 111, ename: 'John', age: 34, salary: 8000 },
  { _id: 112, ename: 'James', age: 35, salary: 8000 },
  { _id: 113, ename: 'Allen', age: 65, salary: 5600 }
]
```

4) Deleting Employee Records

We use following methods to delete record(s)

- a) **deleteOne()** : Deletes a specific record from a table

Example-1 : Deleting an emp record with `_id=110`

```
db.emp.deleteOne({'_id':110});
```

```
mvsr> db.emp.deleteOne({'_id':110})
{ acknowledged: true, deletedCount: 1 }
mvsr> db.emp.find()
[
  { _id: 111, ename: 'John', age: 34, salary: 8000 },
  { _id: 112, ename: 'James', age: 35, salary: 8000 },
  { _id: 113, ename: 'Allen', age: 65, salary: 5600 }
]
```

- b) **deleteMany()** : Deletes all records from a table

Example-2: Delete all records from emp table

```
db.emp.deleteMany();
```

```
mvsr> db.emp.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
mvsr> db.emp.find()
mvsr>
```